

Задача 5 (Познакомиться с pgbench и нагрузочным тестированием)

1. Установить pgbench (входит в состав PostgreSQL).

```
which pgbench
```

```
/home/kitty/postgres/install/debug/bin/pgbench
```

```
pgbench --version
```

```
pgbench (PostgreSQL) 18.4
```

2. Инициализировать тестовую БД: pgbench -i -s 50.

```
createdb pgbench_test
```

```
pgbench -i -s 50 pgbench_test
```

```
dropping old tables ...
```

```
creating tables ...
```

```
generating data (client-side) ...
```

```
vacuuming ...
```

```
creating primary keys ...
```

```
done in 16.46 s (drop tables 0.06 s, create  
tables 0.02 s, client-side generate 10.26 s,  
vacuum 0.80 s, primary keys 5.33 s).
```

3. Произвести базовый запуск:

```
pgbench -c 10 -j 2 -t 1000 -U kitty pgbench_test
```

```
pgbench (18.4)
```

```
starting vacuum ... end.
```

```
transaction type: <builtin: TPC-B (sort of)>
```

```
scaling factor: 50
```

```
query mode: simple
```

```
number of clients: 10
```

```
number of threads: 2
```

```
maximum number of tries: 1
```

```
number of transactions per client: 1000
```

```
number of transactions actually processed:  
10000/10000
```

number of failed transactions: 0 (0.000%)
latency average = 6.477 ms
initial connection time = 13.764 ms
tps = 1543.964227 (without initial connection time)

4. Настройка конфигурации сервера:

Поменяла параметр `shared_buffers` на 500MB, раскомментировала параметр `shared_preload_libraries`, поменяла количество подключений `max_connections` на 130 (чтобы 128 клиентов участвовали в статистике), перезапустила сервер.

5. Подготовка к тестированию.

В задаче нам нужно прогнать `pgbench` дважды - с включенным и выключенным расширением для замера разницы в производительности (я использовала `pg_stat_statements`), поэтому предварительно собрала его. Включение и выключение расширения при тестировании описано в скрипте (см. [Приложение №1](#)).

```
cd ~/postgres/source/debug/contrib/pg_stat_statements
make USE_PGXS=1
PG_CONFIG=$HOME/postgres/install/debug/bin/pg_config
make install USE_PGXS=1
PG_CONFIG=$HOME/postgres/install/debug/bin/pg_config
```

6. Написание скрипта для автоматизации.

Что делает скрипт:

1) Перебирает 4 комбинации:

- 500MB, ext=on
- 500MB, ext=off
- 1GB, ext=on
- 1GB, ext=off

2) Для каждой комбинации:

- Меняет `postgresql.conf` через `sed` (значения `shared_buffers` и `shared_preload_libraries`).
- Перезапускает сервер (`pg_ctl restart`).
- Создаёт/удаляет расширение в БД (`CREATE EXTENSION / DROP EXTENSION`).
- Проверяет через `SHOW`, что параметры применились

3) Для каждой комбинации $\times 8$ клиентов $\times 3$ повтора:

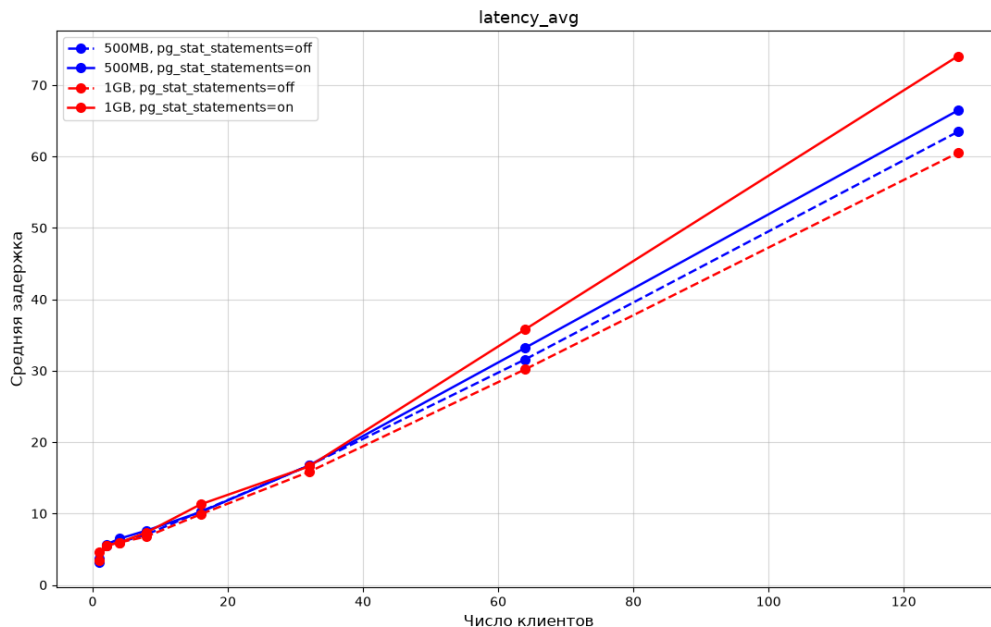
- Запускает `pgbench -c <clients> -j 2 -T 60`.
- Извлекает `tps` и `latency_avg` из вывода через `grep + awk`.
- Пишет строку в CSV `tps_results_full.csv`

Сам скрипт см. в [Приложении №1](#).

7. Обработка результатов.

Обрабатывать результаты из файла `csv` я решила с помощью `Jupyter notebook`, т.к. показалось, что это удобнее (график выводится сразу при исполнении, редактировать удобно без сохранения лишнего). Файл `.ipynb` с построением графиков прикладываю отдельно, если понадобится.

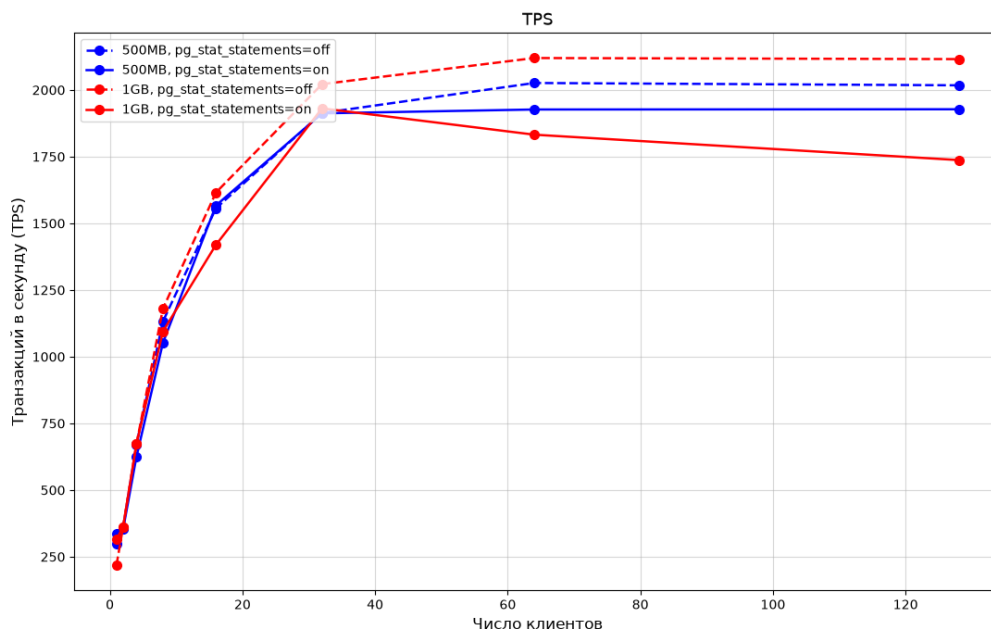
8. Вывод (графики представлены).



Здесь мы можем увидеть, что рост `latency` почти линейный. Пока сервер обслуживает одних клиентов, другие ждут. Чем больше клиентов — тем длиннее очередь — тем выше `latency`.

1GB даёт меньшую `latency`, чем 500MB (ext off), поскольку база данных (750 МБ) полностью помещается в кеш. Разница — ~5% при 128 клиентах.

Включение `pg_stat_statements` увеличивает `latency` на 5–15%. При 1GB эффект сильнее, поскольку расширение вызывается чаще на каждую транзакцию.



До 32 клиентов все четыре линии растут практически одинаково резким скачком — TPS увеличивается с ~300 до ~1900.

После 32 клиентов линии расходятся:

- 1GB, off (красный пунктир) — продолжает расти до ~2100 TPS и выходит на плато.
- 500MB, off (синий пунктир) — плато на ~1950 TPS.
- 500MB, on (синяя сплошная) — плато на ~1920 TPS.
- 1GB, on (красная сплошная) — падает до ~1730 TPS при 128 клиентах.

Резкий рост до 32 клиентов объясняется тем, что сервер не загружен и каждое новое подключение добавляет параллелизма: процессор и диск используются эффективнее и tps растет пропорционально числу клиентов. Дальше уже все ядра процессора загружены и поэтому клиенты просто стоят в очереди (поэтому на графике плато).

Можем также заметить, что tps выше с выключенным расширением, т.к. оно перехватывает и работает с каждым запросом.

Увеличение `shared_buffers` с 500 МБ до 1 ГБ даёт прирост TPS до ~10 процентов при большом числе клиентов, поскольку база данных (750 МБ) перестаёт помещаться в 500 МБ и полностью помещается в 1 ГБ.

Оптимальная конфигурация — та, где TPS максимален, а latency минимальна.

По результатам эксперимента 1GB, off — лучший: TPS ~2100, latency ~60 мс при 128 клиентах.

Приложение к задаче №1 (скрипт для тестирования)

```
#!/bin/bash
# =====
# Полный эксперимент:
#   shared_buffers × pg_stat_statements × clients
# =====

# ----- Настройки -----
DB="pgbench_test"
PGUSER="kitty"
TIME=60
THREADS=2

BUFFERS_SIZES=("500MB" "1GB")
EXT_MODES=("on" "off")
CLIENTS=(1 2 4 8 16 32 64 128)
REPEATS=3

DATA_DIR="$HOME/postgres/data/debug"
LOG_DIR="$HOME/postgres/log/debug"
INSTALL_DIR="$HOME/postgres/install/debug"
PG_CTL="$INSTALL_DIR/bin/pg_ctl"
PGBENCH="$INSTALL_DIR/bin/pgbench"
PSQL="$INSTALL_DIR/bin/psql"

RESULT_FILE="tps_results_full.csv"
TEMP_DIR="temp_results_full"

# ----- Подготовка -----
mkdir -p "$TEMP_DIR"
echo "buffers,mode,clients,run,tps,latency_avg" >
"$RESULT_FILE"
```

```
# ----- Функция: настройка конфига -----
set_config() {
    local buffers=$1
    local mode=$2      # "on" или "off"

    echo
    "===== "
    echo "Настройка: shared_buffers = $buffers,
pg_stat_statements = $mode"
    echo
    "===== "

    # shared_buffers
    sed -i.bak "s/^shared_buffers =
./shared_buffers = $buffers/"
"$DATA_DIR/postgresql.conf"

    # shared_preload_libraries
    if [ "$mode" = "on" ]; then
        sed -i.bak "s|^#*shared_preload_libraries
= .*|shared_preload_libraries =
'pg_stat_statements'|" "$DATA_DIR/postgresql.conf"
    else
        sed -i.bak "s|^#*shared_preload_libraries
= .*|shared_preload_libraries = ''|"
"$DATA_DIR/postgresql.conf"
    fi

    # Перезапуск
    $PG_CTL -D "$DATA_DIR" -l
"$LOG_DIR/postgres.log" restart
    sleep 3

    # Создать/удалить расширение в базе
    if [ "$mode" = "on" ]; then
        $PSQL -U "$PGUSER" -d "$DB" -c "CREATE
EXTENSION IF NOT EXISTS pg_stat_statements;"
    >/dev/null 2>&1
}
```

```
        else
            $PSQL -U "$PGUSER" -d "$DB" -c "DROP
EXTENSION IF EXISTS pg_stat_statements;"
>/dev/null 2>&1
        fi
```

Проверка

```
        echo "Проверка: shared_buffers = $($PSQL -U
"$PGUSER" -d postgres -t -c 'SHOW
shared_buffers;' | tr -d ' ')"
        echo "Проверка: shared_preload_libraries =
$($PSQL -U "$PGUSER" -d postgres -t -c 'SHOW
shared_preload_libraries;' | tr -d ' ')"
    }
```

————— Функция: один прогон —————

```
run_and_extract() {
    local buffers=$1
    local mode=$2
    local clients=$3
    local run=$4

    local
temp_log="$TEMP_DIR/bench_${buffers}_${mode}_c${
clients}_run${run}.log"
    echo "Запуск: buffers=$buffers, ext=$mode,
clients=$clients, run=$run"

    $PGBENCH -c $clients -j $THREADS -T $TIME -U
"$PGUSER" $DB > "$temp_log" 2>&1

    local tps=$(grep "tps =" "$temp_log" | head -
1 | awk '{print $3}')
    local latency_avg=$(grep "latency average ="
"$temp_log" | awk '{print $4}')

    [ -z "$tps" ] && tps="ERROR"
    [ -z "$latency_avg" ] && latency_avg="ERROR"
```

```
        echo
"$buffers,$mode,$clients,$run,$tps,$latency_avg"
>> "$RESULT_FILE"
        echo "    → TPS: $tps, latency: $latency_avg
ms"
    }
```

————— Основной цикл —————

```
echo "Начинаем полное тестирование (96
прогонов)"
```

```
echo "Результаты: $RESULT_FILE"
```

```
echo ""
```

```
for buffers in "${BUFFERS_SIZES[@]}; do
    for mode in "${EXT_MODES[@]}; do
        set_config "$buffers" "$mode"
        for clients in "${CLIENTS[@]}; do
            for run in $(seq 1 $REPEATS); do
                run_and_extract "$buffers"
"$mode" "$clients" "$run"
            done
        done
    done
done
```

```
echo ""
```

```
echo "=====
```

```
echo "Все тесты завершены!"
```

```
echo "Результаты: $RESULT_FILE"
```

```
echo "=====
```

```
head -10 "$RESULT_FILE"
```